WhatWeEatInAmerica

August 4, 2026

1 Stratifying "What we eat in America"

We apply Dleto stratification on tensors emerging from nutrition data. We use a USDA Government survey

> What We Eat in America, NHANES 2017-March 2020 Prepandemic

The data consists of several thousand foods sold to the public within the USA with each item assigned measures of multiple food types. For example, what fraction of a serving consists of real fruits, vegetables, grains, added sugars and etc.

> **DISCLAIMER.** This notebook is intended to illustrate Dleto stratification algorithms only and does not represent any suggested or implied nutritional advise nor does it represent a complete understanding of the underlying significance of the data.

1.1 1. Loading the data

We begin by loading the data and preparing for analysis.

Let us start by loading the package we will use. In some situations you may need to add the package to your Julia system by uncommenting the relevant commands. This is typically a one-time step and can be avoided in future runs.

[1]:
```
using Pkg
# Uncomment to install packages if needed.
Pkg.add("CSV"); Pkg.add("DataFrames")
using CSV, DataFrames
Pkg.activate("../")  # Activate the main project
using Dleto
using Plots
```

```
   Resolving package versions…
      Compat entries added for
     Project No packages added to or removed from
`~/CODE/OpenDleto/Project.toml`
    Manifest No packages added to or removed from
`~/CODE/OpenDleto/Manifest.toml`
   Resolving package versions…
      Compat entries added for
     Project No packages added to or removed from
`~/CODE/OpenDleto/Project.toml`
    Manifest No packages added to or removed from
`~/CODE/OpenDleto/Manifest.toml`
  Activating project at `~/CODE/OpenDleto`
```

```
WebIO._IJuliaInit()
```

```
Loading Dleto Plots Extension
```

Note. The USDA data set is provided as a single Microsoft Excel formatted spreadsheet consisting of two sheets: the data and a key. For convenience the file is stored locally as two separate CSV files, one for each sheet.

[2]:
```
# Load the data and key files
data = CSV.read("FPED_1720.csv", DataFrame)
key_data = CSV.read("FPED_1720-key.csv", DataFrame)

println("Total numer of foods: $(size(data,1))")
println("Number of Categories: $(size(key_data,1))")
println("\nSample of food items:")
println(data[147:155, vcat(1:2, 33:34, 38)])  # Show first few items and columns
```

```
Total numer of foods: 7444
Number of Categories: 39

Sample of food items:
9×5 DataFrame
 Row   FOODCODE  DESCRIPTION
D_MILK (cup eq)  D_YOGURT (cup eq)  ADD_SUGARS (tsp eq)
       Int64     String
 Float64          Float64            Float64




   1   11513802  Chocolate milk, made from light …             0.36
0.0                  0.85
   2   11513803  Chocolate milk, made from light …             0.36
0.0                  0.85
   3   11513804  Chocolate milk, made from light …             0.36
```

```
0.0                 0.85
   4   11513805  Chocolate milk, made from light …            0.18
0.0                 1.61
   5   11513850  Chocolate milk, made from sugar …            0.36
0.0                 0.0
   6   11513851  Chocolate milk, made from sugar …            0.36
0.0                 0.0
   7   11513852  Chocolate milk, made from sugar …            0.36
0.0                 0.0
   8   11513853  Chocolate milk, made from sugar …            0.36
0.0                 0.0
   9   11513854  Chocolate milk, made from sugar …            0.36
0.0                 0.0
```

More detail about the labels may be obtained by inspecting them individually. For instance, a food product that includes "light syrup" has sugar as an added ingredient, whereas "sugar free syrup" adds 0.0 sugar.

[3]:
```
println( data[147,2])
println( data[155,2])
```

```
Chocolate milk, made from light syrup with reduced fat milk
Chocolate milk, made from sugar free syrup with fat free milk
```

To understand the 39 categories we can access the `key_data`.

[4]:
```
# Examine the key data to understand the nutritional categories
println("Key data structure:")
println(key_data)
```

```
Key data structure:
39×4 DataFrame
 Row   column   variable_name

variable_description                 units
      String3  String31          String

 String15?


   1   A        FOODCODE          Food code

missing
   2   B        DESCRIPTION       Food description

missing
   3   C        F_TOTAL           Total intact or cut fruits and f…  cup eq
   4   D        F_CITMLB          Intact fruits (whole or cut) of …  cup eq
   5   E        F_OTHER           Intact fruits (whole or cut); ex…  cup eq
   6   F        F_JUICE           Fruit juices, citrus and non cit…  cup eq
   7   G        V_TOTAL           Total dark green, red and orange…  cup eq
```

```
  8   H        V_DRKGR           Dark green vegetables              cup eq
  9   I        V_REDOR_TOTAL     Total red and orange vegetables …  cup eq
 10   J        V_REDOR_TOMATO    Tomatoes and tomato products       cup eq
 11   K        V_REDOR_OTHER     Other red and orange vegetables,…  cup eq
 12   L        V_STARCHY_TOTAL   Total starchy vegetables (white …  cup eq
 13   M        V_STARCHY_POTATO  White potatoes                    cup eq
 14   N        V_STARCHY_OTHER   Other starchy vegetables, exclud…  cup eq
 15   O        V_OTHER           Other vegetables not in the vege…  cup eq
 16   P        V_LEGUMES         Legumes computed as vegetables     cup eq
 17   Q        G_TOTAL           Total whole and refined grains     oz eq
 18   R        G_WHOLE           Whole grains                       oz eq
 19   S        G_REFINED         Refined or non-whole grains        oz eq
 20   T        PF_TOTAL          Total meat, poultry, seafood, or…  oz eq
 21   U        PF_MPS_TOTAL      Total meat, poultry, seafood, or…  oz eq
 22   V        PF_MEAT           Beef, veal, pork, lamb, game mea…  oz eq
 23   W        PF_CUREDMEAT      Cured/luncheon meat made from be…  oz eq
 24   X        PF_ORGAN          Organ meat from beef, veal, pork…  oz eq
 25   Y        PF_POULT          Chicken, turkey, Cornish hens, a…  oz eq
 26   Z        PF_SEAFD_HI       Seafood (finfish, shellfish and …  oz eq
 27   AA       PF_SEAFD_LOW      Seafood (finfish, shellfish and …  oz eq
 28   AB       PF_EGGS           Eggs (chicken, duck, goose, quai…  oz eq
 29   AC       PF_SOY            Soy products, excluding calcium …  oz eq
 30   AD       PF_NUTSDS         Peanuts, tree nuts, and seeds, e…  oz eq
 31   AE       PF_LEGUMES        Legumes computed as protein foods  oz eq
 32   AF       D_TOTAL           Total milk, yogurt, cheese, and …  cup eq
 33   AG       D_MILK            Fluid milk and calcium fortified…  cup eq
 34   AH       D_YOGURT          Yogurt                             cup eq
 35   AI       D_CHEESE          Cheese                             cup eq
 36   AJ       OILS              Oils                               grams
 37   AK       SOLID_FATS        Solid fats                         grams
 38   AL       ADD_SUGARS        Foods defined as added sugars      tsp eq
 39   AM       A_DRINKS          Alcoholic beverages                no. of
drinks
```

To prepare for further analysis, we do an initial analysis that identifies missing values and total range of values.

[5]:
```
using Statistics

# Extract the numerical data for tensor creation
# Remove the first two columns (FOODCODE and DESCRIPTION) to get only numerical␣
 ↪data
numerical_data = Matrix(data[:, 3:end])

println("Numerical data shape: $(size(numerical_data))")
println("Data type: $(eltype(numerical_data))")

# Check for any missing values
```

```
missing_count = sum(ismissing.(numerical_data))
println("Missing values: $missing_count")

# Summary statistics
println("\nData range:")
println("Min value: $(minimum(skipmissing(numerical_data)))")
println("Max value: $(maximum(skipmissing(numerical_data)))")
println("Mean value: $(mean(skipmissing(numerical_data)))")

# Check if data is sparse (many zeros)
zero_count = sum(numerical_data .== 0)
total_elements = length(numerical_data)
sparsity = zero_count / total_elements
println("Sparsity (fraction of zeros): $(round(sparsity, digits=3))")
```

```
Numerical data shape: (7444, 37)
Data type: Float64
Missing values: 0

Data range:
Min value: 0.0
Max value: 100.0
Mean value: 0.329505351670854
Sparsity (fraction of zeros): 0.831
```

This data is carefully constructed and has no missing values. Many values are 0.0, however, which may indicate those measurements are below some threshold.

1.2 2. Creating food categories

We next perform an analysis of this data set that explores general categories of nutrition as supplied in the What We Eat in America Food Categories. We consolidate some of the 15 categories identified there, using instead the following 7 slightly courser categories: - FRUITS: citrus/melons/berries, other fruits, and fruit juice - VEGETABLES: color-based groupings (dark green, red/orange), starchy vegetables (potatoes vs others), legumes, and other vegetables - GRAINS: whole grains and refined grains - PROTEIN FOODS: meat, poultry, seafood (high/low omega-3), eggs, soy, nuts/seeds, and legumes - DAIRY: milk, yogurt, and cheese - FATS & OILS: liquid oils and solid fats - DISCRETIONARY CALORIES: added sugars and alcoholic beverages

An inspection of the column headings above shows the fruit related columns are indexed by `F_` terms, vegetables by `V_`, grains by `G_` and so forth. However, there is also a cumulative column `F_Total` that may be excluded as it is derived from the other columns.

Let us create these categories for our computation.

[6]:
```
nutritional_columns = names(data)[3:end]  # Skip FOODCODE and DESCRIPTION

# Define natural category groupings based on USDA food patterns
isfruit(col) = startswith(string(col), "F_") && !occursin("TOTAL", string(col))
```

```
fruits = filter(isfruit, nutritional_columns)

println("\n FRUITS ($(length(fruits)) categories):")
for (i, fruit) in enumerate(fruits)
    println("  $i. $fruit")
end
```

```
 FRUITS (3 categories):
  1. F_CITMLB (cup eq)
  2. F_OTHER (cup eq)
  3. F_JUICE (cup eq)
```

[7]:
```
isvegetable(col) = startswith(string(col), "V_") && !occursin("TOTAL",␣
 ↪string(col))
vegetables = filter(isvegetable, nutritional_columns)
println("\n VEGETABLES ($(length(vegetables)) categories):")
for (i, veg) in enumerate(vegetables)
    println("  $i. $veg")
end
```

```
 VEGETABLES (7 categories):
  1. V_DRKGR (cup eq)
  2. V_REDOR_TOMATO (cup eq)
  3. V_REDOR_OTHER (cup eq)
  4. V_STARCHY_POTATO (cup eq)
  5. V_STARCHY_OTHER (cup eq)
  6. V_OTHER (cup eq)
  7. V_LEGUMES (cup eq)
```

[8]:
```
isgrain(col) = startswith(string(col), "G_") && !occursin("TOTAL", string(col))
grains       = filter(isgrain, nutritional_columns)
println("\n GRAINS ($(length(grains)) categories):")
for (i, grain) in enumerate(grains)
    println("  $i. $grain")
end
```

```
 GRAINS (2 categories):
  1. G_WHOLE (oz eq)
  2. G_REFINED (oz eq)
```

[9]:
```
isprotein(col) = startswith(string(col), "PF_") && !occursin("TOTAL",␣
 ↪string(col))
proteins = filter(isprotein, nutritional_columns)
println("\n PROTEIN FOODS ($(length(proteins)) categories):")
for (i, protein) in enumerate(proteins)
```

```
    println("  $i. $protein")
end
```

```
 PROTEIN FOODS (10 categories):
  1. PF_MEAT (oz eq)
  2. PF_CUREDMEAT (oz eq)
  3. PF_ORGAN (oz eq)
  4. PF_POULT (oz eq)
  5. PF_SEAFD_HI (oz eq)
  6. PF_SEAFD_LOW (oz eq)
  7. PF_EGGS (oz eq)
  8. PF_SOY (oz eq)
  9. PF_NUTSDS (oz eq)
  10. PF_LEGUMES (oz eq)
```

[10]:
```
isdairy(col) = startswith(string(col), "D_") && !occursin("TOTAL", string(col))
dairy        = filter(isdairy, nutritional_columns)
println("\n DAIRY ($(length(dairy)) categories):")
for (i, d) in enumerate(dairy)
    println("  $i. $d")
end
```

```
 DAIRY (3 categories):
  1. D_MILK (cup eq)
  2. D_YOGURT (cup eq)
  3. D_CHEESE (cup eq)
```

[11]:
```
isfat_or_oil(col) = string(col) in ["OILS (grams)", "SOLID_FATS (grams)"]
fats_oils     = filter(isfat_or_oil, nutritional_columns)
println("\n FATS & OILS ($(length(fats_oils)) categories):")
for (i, fat) in enumerate(fats_oils)
    println("  $i. $fat")
end
```

```
 FATS & OILS (2 categories):
  1. OILS (grams)
  2. SOLID_FATS (grams)
```

[12]:
```
isdiscretionary(col) = string(col) in ["ADD_SUGARS (tsp eq)", "A_DRINKS (no. of␣
 ↪drinks)"]
discretionary = filter(isdiscretionary, nutritional_columns)
println("\n DISCRETIONARY CALORIES ($(length(discretionary)) categories):")
for (i, disc) in enumerate(discretionary)
    println("  $i. $disc")
end
```

```
 DISCRETIONARY CALORIES (2 categories):
  1. ADD_SUGARS (tsp eq)
  2. A_DRINKS (no. of drinks)
```

[13]:
```
println("SUMMARY OF THE 7 NUTRITIONAL CATEGORIES:")
println("="^50)
println("  FRUITS ($(length(fruits))):")
println("   - Citrus/melons/berries, other fruits, fruit juice (excluding␣
 ↪totals)")

println("\n  VEGETABLES ($(length(vegetables))):")
println("   - Dark green, red/orange, starchy (potatoes/others), legumes, other␣
 ↪vegetables (excluding totals)")

println("\n  GRAINS ($(length(grains))):")
println("   - Whole grains and refined grains (excluding totals)")

println("\n  PROTEIN FOODS ($(length(proteins))):")
println("   - Meat, poultry, seafood (high/low omega-3), eggs, soy, nuts/seeds,␣
 ↪legumes (excluding totals)")

println("\n  DAIRY ($(length(dairy))):")
println("   - Milk, yogurt, cheese (excluding totals)")

println("\n  FATS & OILS ($(length(fats_oils))):")
println("   - Liquid oils and solid fats")

println("\n  DISCRETIONARY CALORIES ($(length(discretionary))):")
println("   - Added sugars and alcoholic beverages")
```

```
SUMMARY OF THE 7 NUTRITIONAL CATEGORIES:
==================================================
  FRUITS (3):
   - Citrus/melons/berries, other fruits, fruit juice (excluding totals)

  VEGETABLES (7):
   - Dark green, red/orange, starchy (potatoes/others), legumes, other vegetables
(excluding totals)

  GRAINS (2):
   - Whole grains and refined grains (excluding totals)

  PROTEIN FOODS (10):
   - Meat, poultry, seafood (high/low omega-3), eggs, soy, nuts/seeds, legumes
(excluding totals)

  DAIRY (3):
```

```
 - Milk, yogurt, cheese (excluding totals)

FATS & OILS (2):
 - Liquid oils and solid fats

DISCRETIONARY CALORIES (2):
 - Added sugars and alcoholic beverages
```

1.3 3. A "What-We-Eat" tensor

There are many ways to build a tensor from these data. As illustration we explore the relationship of added sugars to products that range over the categories of Fruits, Vegetables, and Proteins. This produces a 3-way tensor with modes `Fruit x Vegetables x Protein`, where very entry is the total amount of added sugars for food groups in that category.

Since the data is very sparse, many entries score `0.0` in one or more of the mode categories. In that case we add a special `Empty` class to each of these categories to indicate that the food group does not contain any measureable amount of the specified category. For instance, milk may contain `0.0` detectable Fruit so it would be placed in the `Empty` fruit category.

[14]:
```
# Use the specific categories but add "empty" options
fruits_with_empty = vcat(fruits, ["F_EMPTY"])
vegetables_with_empty = vcat(vegetables, ["V_EMPTY"])
proteins_with_empty = vcat(proteins, ["PF_EMPTY"])

println("(#Fruits, #Vegetables, #Proteins) = ($(length(fruits_with_empty)),␣
 ↪$(length(vegetables_with_empty)), $(length(proteins_with_empty)) )")
```

```
(#Fruits, #Vegetables, #Proteins) = (4, 8, 11 )
```

Now we create the tensor with these modes and total added sugar as the value in each entry. We will be using `ITensors` as our backend for tensors. This requires each mode to be created as an `Index` which will prevent the complication of remembering which mode was first, second, or third.

[15]:
```
using ITensors

# Create new tensor with empty categories
f = Index(length(fruits_with_empty), "Fruits")
v = Index(length(vegetables_with_empty), "Veg")
p = Index(length(proteins_with_empty), "Prot")

println("\nIndices: $(f) × $(v) × $(p)")
```

```
Indices: (dim=4|id=439|"Fruits") × (dim=8|id=102|"Veg") × (dim=11|id=337|"Prot")
```

To build the tensor we traverse very row of the data and check for a nonzero added sugar value. Upon encountering such a value we then designate the `(f,v,p)` coordinates of that entry add them to any existing value in the coordinate.

[16]:
```
# Rebuild mode indices if they were shadowed by later notebook variables.
if !(f isa Index)
    f = Index(length(fruits_with_empty), "Fruits")
end
if !(v isa Index)
    v = Index(length(vegetables_with_empty), "Veg")
end
if !(p isa Index)
    p = Index(length(proteins_with_empty), "Prot")
end

Sugar = ITensor(f,v,p)

# Function to explain index meaning
function friendly_label(code::AbstractString)
    label_map = Dict(
        "F_CITMLB (cup eq)" => "Citrus, tomatoes, melons, and berries",
        "F_OTHER (cup eq)" => "Other fruits (apples, bananas, etc.)",
        "F_JUICE (cup eq)" => "Fruit juice",
        "V_DRKGR (cup eq)" => "Dark green vegetables",
        "V_REDOR_TOMATO (cup eq)" => "Red/orange vegetables (tomatoes)",
        "V_REDOR_OTHER (cup eq)" => "Red/orange vegetables (other)",
        "V_STARCHY_POTATO (cup eq)" => "Starchy vegetables (potatoes)",
        "V_STARCHY_OTHER (cup eq)" => "Starchy vegetables (other)",
        "V_OTHER (cup eq)" => "Other vegetables",
        "V_LEGUMES (cup eq)" => "Legumes (as vegetables)",
        "PF_MEAT (oz eq)" => "Meat (beef, pork, lamb, game)",
        "PF_CUREDMEAT (oz eq)" => "Cured meat",
        "PF_ORGAN (oz eq)" => "Organ meats",
        "PF_POULT (oz eq)" => "Poultry",
        "PF_SEAFD_HI (oz eq)" => "High omega-3 seafood",
        "PF_SEAFD_LOW (oz eq)" => "Low omega-3 seafood",
        "PF_EGGS (oz eq)" => "Eggs",
        "PF_SOY (oz eq)" => "Soy products",
        "PF_NUTSDS (oz eq)" => "Nuts and seeds",
        "PF_LEGUMES (oz eq)" => "Legumes (as protein)",
        "F_EMPTY" => "No measurable fruit",
        "V_EMPTY" => "No measurable vegetable",
        "PF_EMPTY" => "No measurable protein"
    )
    return get(label_map, code, code)
end

processed_foods = 0
empty_assignments = Dict(:fruits => 0, :vegetables => 0, :proteins => 0)

for row_idx in 1:nrow(data)
```

```
    sugar_value = data[row_idx, "ADD_SUGARS (tsp eq)"]

    if ismissing(sugar_value) || sugar_value == 0.0
        continue
    end

    # Get vector of fruit, veg, and protein for this food entry.
    fruit_vals = [coalesce(data[row_idx, col], 0.0) for col in fruits]
    veg_vals = [coalesce(data[row_idx, col], 0.0) for col in vegetables]
    protein_vals = [coalesce(data[row_idx, col], 0.0) for col in proteins]

    # Determine indices with empty categories
    if maximum(fruit_vals) <= 1e-10
        fruit_idx = length(fruits_with_empty)
        empty_assignments[:fruits] += 1
    else
        fruit_idx = argmax(fruit_vals)
    end

    if maximum(veg_vals) <= 1e-10
        veg_idx = length(vegetables_with_empty)
        empty_assignments[:vegetables] += 1
    else
        veg_idx = argmax(veg_vals)
    end

    if maximum(protein_vals) <= 1e-10
        protein_idx = length(proteins_with_empty)
        empty_assignments[:proteins] += 1
    else
        protein_idx = argmax(protein_vals)
    end

    # Add sugar value to tensor
    Sugar[f=>fruit_idx, v=>veg_idx, p=>protein_idx] += sugar_value
    processed_foods += 1
end

# Analyze the tensor directly from ITensor entries (no dense Array conversion).
non_zero_empty = 0
max_value = -Inf
for i in 1:ITensors.dim(f), j in 1:ITensors.dim(v), k in 1:ITensors.dim(p)
    entry = Sugar[f=>i, v=>j, p=>k]
    if entry != 0.0
        non_zero_empty += 1
    end
    if entry > max_value
```

```
        max_value = entry
    end
end

total_entries = ITensors.dim(f) * ITensors.dim(v) * ITensors.dim(p)

# Print a summary of the tensor.
println("\n  Tensor with empty categories created!")
println("Foods with added sugars: $processed_foods, $(round((processed_foods /␣
 ↪(size(data,1)-2)) * 100, digits=2))%")
println("Non-zero entries: $non_zero_empty / $total_entries")
println("Sparsity: $(round((total_entries - non_zero_empty) / total_entries *␣
 ↪100, digits=1))%")
println("Max value: $(round(max_value, digits=2)) tsp eq")

println("  Foods with 0 fruits: $(empty_assignments[:fruits]) foods")
println("  Foods with 0 vegetables: $(empty_assignments[:vegetables]) foods")
println("  Foods with 0 proteins: $(empty_assignments[:proteins]) foods")
```

```
  Tensor with empty categories created!
Foods with added sugars: 3194, 42.92%
Non-zero entries: 91 / 352
Sparsity: 74.1%
Max value: 4210.98 tsp eq
  Foods with 0 fruits: 2708 foods
  Foods with 0 vegetables: 2471 foods
  Foods with 0 proteins: 1424 foods
```

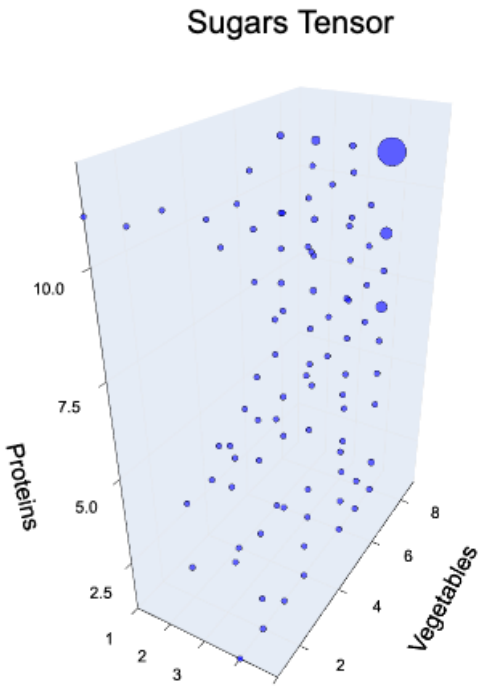
[17]:
```
# Mode-label helpers: use explicit dimension convention first.
# User convention for this notebook: 4=fruit, 8=veg, 11=protein.
function mode_key(ix::Index)
    d = ITensors.dim(ix)
    if d == 4
        return :fruit
    elseif d == 8
        return :veg
    elseif d == 11
        return :protein
    end

    # Fallback for any future tensors that don't follow 4/8/11.
    txt = lowercase(string(tags(ix)) * " " * string(id(ix)))
    if occursin("fruit", txt)
        return :fruit
    elseif occursin("veg", txt) || occursin("vegetable", txt)
        return :veg
```

```
    elseif occursin("prot", txt) || occursin("protein", txt)
        return :protein
    else
        return :unknown
    end
end

function mode_label(ix::Index; mixed=false)
    m = mode_key(ix)
    if m == :fruit
        return mixed ? "Fruit mix" : "Fruits"
    elseif m == :veg
        return mixed ? "Vegetable mix" : "Vegetables"
    elseif m == :protein
        return mixed ? "Protein mix" : "Proteins"
    else
        return mixed ? "Mixed mode" : "Mode"
    end
end
```

[17]:
```
mode_label (generic function with 1 method)
```

[32]:
```
inds_sugar = collect(ITensors.inds(Sugar))
plot_tensor(Sugar, title="Sugars Tensor",
        xlabel=mode_label(inds_sugar[1]), ylabel=mode_label(inds_sugar[2]),␣
 ↪zlabel=mode_label(inds_sugar[3]))
```

[32]:



This data shows that 42% of the foods consumed in the USA for this study contained added sugars. We can see this some what sparse and may have some clustering but it is spread out and perhaps just an optical illusion. We will tart to look at the tensor with existing tools then turn to Dleto's `stratify` operations to see if we can detect the types of products that use the most added sugars.

1.4 3.1 Sample the Tensor

For instance, our tensors 1,1,1 entry is the total added sugars for food products that have 1 serving of fruit X, Protein Y, and Vegetable Z.

[19]:
```
# Human-readable interpretation of the (1,1,1) sugar tensor entry
fruit_code = fruits_with_empty[1]
veg_code = vegetables_with_empty[1]
protein_code = proteins_with_empty[1]

label_map = Dict(
    "F_CITMLB (cup eq)" => "Citrus, tomatoes, melons, and berries",
    "V_DRKGR (cup eq)" => "Dark green vegetables",
    "PF_MEAT (oz eq)" => "Meat (beef, pork, lamb, game)"
)

fruit_label = get(label_map, fruit_code, fruit_code)
```

```
veg_label = get(label_map, veg_code, veg_code)
protein_label = get(label_map, protein_code, protein_code)

sugar_tsp = Sugar[f=>1, v=>1, p=>1]
println("(1,1,1) corresponds to: $fruit_label + $veg_label + $protein_label")
println("Total added sugar in this bucket: $(round(sugar_tsp, digits=2)) tsp␣
 ↪eq")
```

```
(1,1,1) corresponds to: Citrus, tomatoes, melons, and berries + Dark green
vegetables + Meat (beef, pork, lamb, game)
Total added sugar in this bucket: 0.0 tsp eq
```

Let us find an entry that is nonzero

[20]:
```
idx = nothing
val = 0.0

for i in 1:dim(f), j in 1:dim(v), k in 1:dim(p)
    entry = Sugar[f=>i, v=>j, p=>k]
    if entry > 0
        idx = (i, j, k)
        val = entry
        break
    end
end

if isnothing(idx)
    println("No nonzero entries found in Sugar.")
else
    i, j, k = idx

    fruit_code = fruits_with_empty[i]
    veg_code = vegetables_with_empty[j]
    protein_code = proteins_with_empty[k]

    println("Found nonzero entry at (i,j,k) = ($i,$j,$k)")
    println("Meaning: $(friendly_label(fruit_code)) +␣
 ↪$(friendly_label(veg_code)) + $(friendly_label(protein_code))")
    println("Total added sugar in this bucket: $(round(val, digits=2)) tsp eq")
end
```

```
Found nonzero entry at (i,j,k) = (1,1,11)
Meaning: Citrus, tomatoes, melons, and berries + Dark green vegetables + No
measurable protein
Total added sugar in this bucket: 0.4 tsp eq
```

1.5 3.2 Look for Tucker decompositions.

Now let us use Dleto. We start by asking considering a Tucker decomposition, also known as a degeneracy. In the original data the Tucker decompositions are all trivial up the default tolerances.

[21]:
```
# Apply Dleto tensor analysis: nondeg and stratify
using Dleto

nondeg_Sugar, Ys = nondeg(Sugar)
# println("\nNondegenerate tensor created with $(nondeg_Sugar) non-zero␣
 ↪entries")

# Compare original index dimensions to nondegenerate index dimensions␣
 ↪(order-invariant).
orig_inds = collect(ITensors.inds(Sugar))
nondeg_inds = collect(ITensors.inds(nondeg_Sugar))

orig_dims = [ITensors.dim(i) for i in orig_inds]
nondeg_dims = [ITensors.dim(i) for i in nondeg_inds]

sorted_orig_dims = sort(orig_dims)
sorted_nondeg_dims = sort(nondeg_dims)
dim_diffs = sorted_orig_dims .- sorted_nondeg_dims

println("Sorted original dims: $sorted_orig_dims")
println("Sorted nondeg dims:   $sorted_nondeg_dims")
println("Dimension differences after sorting (orig - nondeg): $dim_diffs")
```

```
Sorted original dims: [4, 8, 11]
Sorted nondeg dims:   [4, 8, 11]
Dimension differences after sorting (orig - nondeg): [0, 0, 0]
Sorted nondeg dims:   [4, 8, 11]
Dimension differences after sorting (orig - nondeg): [0, 0, 0]
```

Tucker decompositions were not informative.

1.6 3.3 Look for a HOSVD structure.

Next we look to see what is signaled by the higher-order singular values.

[22]:
```
using LinearAlgebra

# ITensor-native HOSVD on nondeg_Sugar (no dense Array conversion).
T_hosvd = nondeg_Sugar
mode_inds = collect(ITensors.inds(T_hosvd))
N = length(mode_inds)

U_factors = Vector{ITensor}(undef, N)
link_inds = Vector{Index}(undef, N)
```

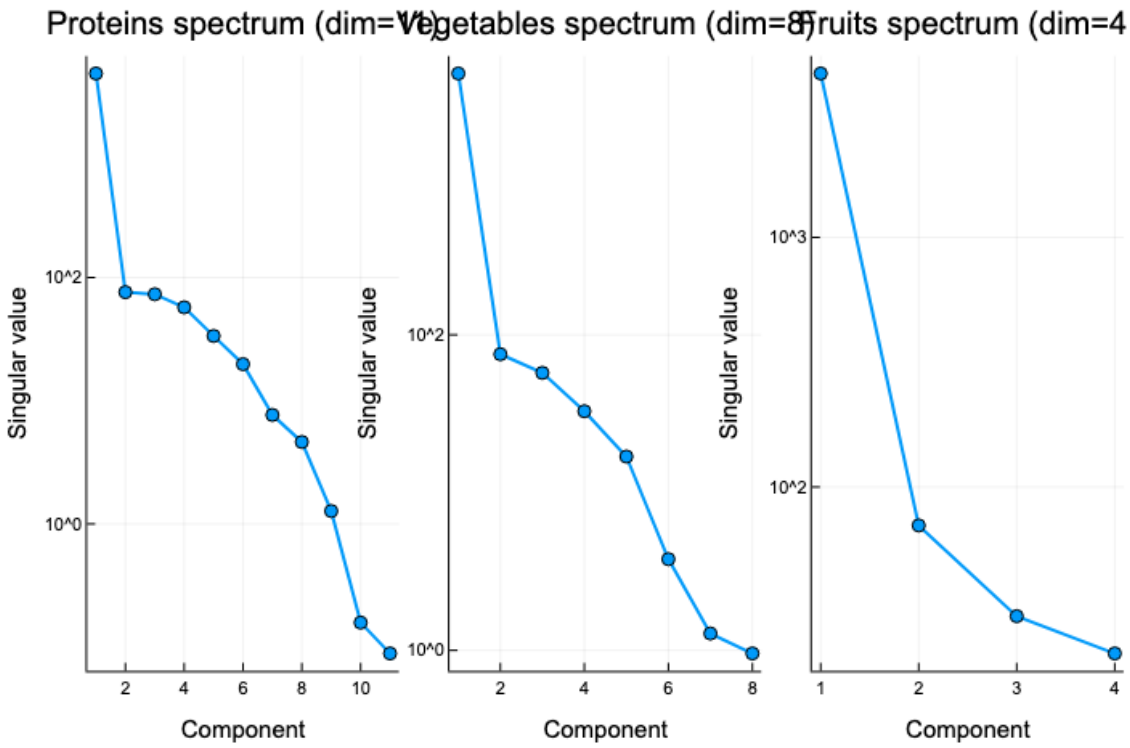
```
svs = Vector{Vector{Float64}}(undef, N)

for n in 1:N
    U, S, V = ITensors.svd(T_hosvd, mode_inds[n])
    U_factors[n] = U

    s1, s2 = ITensors.inds(S)
    link_inds[n] = s1
    svs[n] = [S[s1=>k, s2=>k] for k in 1:ITensors.dim(s1)]
end

# Core tensor G = T ×1 U1' ×2 U2' ... ×N UN'
G_it = T_hosvd
for n in 1:N
    G_it *= dag(U_factors[n])
end

# Reconstruct from HOSVD and report relative error.
A_hat_it = G_it
for n in 1:N
    A_hat_it *= U_factors[n]
end
rel_err = norm(T_hosvd - A_hat_it) / max(norm(T_hosvd), eps(Float64))

orig_dims = [ITensors.dim(i) for i in mode_inds]
core_dims = [ITensors.dim(i) for i in ITensors.inds(G_it)]
factor_dims = [(ITensors.dim(mode_inds[n]), ITensors.dim(link_inds[n])) for n␣
 ↪in 1:N]

println("HOSVD computed for order-$N tensor.")
println("Original dims: $orig_dims")
println("Core dims:     $core_dims")
println("Factor dims (mode, rank): $factor_dims")
println("Relative reconstruction error: $(rel_err)")
```

```
HOSVD computed for order-3 tensor.
Original dims: [11, 8, 4]
Core dims:     [11, 8, 4]
Factor dims (mode, rank): [(11, 11), (8, 8), (4, 4)]
Relative reconstruction error: 9.526529123620498e-16
```

Plot the higher order singular values in each mode and then the core tensor of the highest value.

[23]:
```
# Plot HOSVD spectra from ITensor singular values (mode unfoldings).
p = plot(layout=(1, N), size=(420 * N, 320), legend=false)
for n in 1:N
    plot!(
        p[n],
```

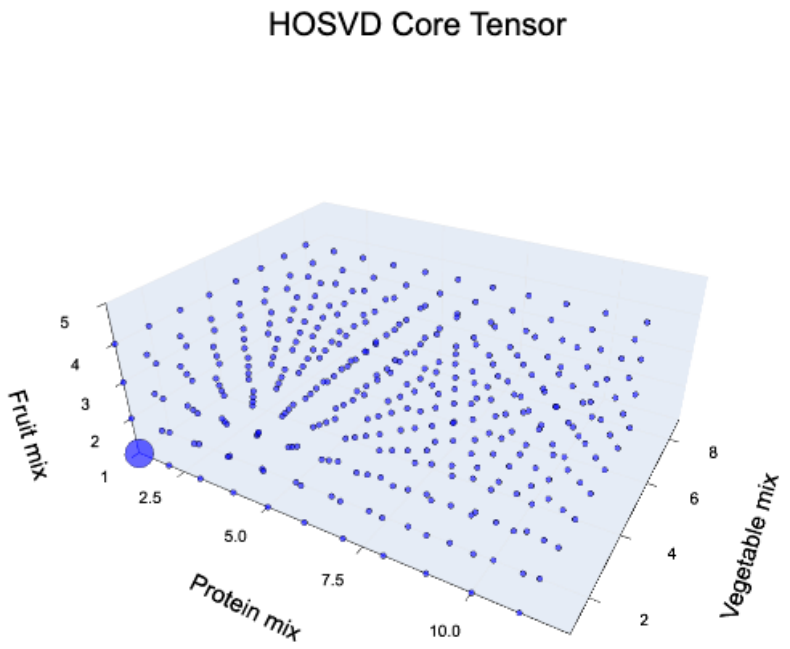
```
        1:length(svs[n]),
        max.(svs[n], eps(Float64)),
        marker=:circle,
        linewidth=2,
        xlabel="Component",
        ylabel="Singular value",
        yscale=:log10,
        title="$(mode_label(mode_inds[n])) spectrum (dim=$(orig_dims[n]))"
    )
end

display(p)
```



[24]:
```
# Plot the tensor transformed by HOSVD (the core tensor G) with labels from␣
 ↪index names.
inds_core = collect(ITensors.inds(G_it))
plot_tensor(
    G_it,
    title="HOSVD Core Tensor",
    xlabel=mode_label(inds_core[1]; mixed=true),
    ylabel=mode_label(inds_core[2]; mixed=true),
```

```
    zlabel=mode_label(inds_core[3]; mixed=true)
)
```

[24]:



1.7 3.4 ParaFAC decomposition.

Last we look for a feature in low-rank approximation techniques, specifically ParaFAC.

[25]:

```
using Random, LinearAlgebra

# PARAFAC / CP-ALS decomposition of the sugar tensor.
# We use a rank-R model and reconstruct the approximating tensor for display.
@isdefined(Sugar) || error("Sugar is not defined. Run the sugar tensor␣
 ↪construction cells first.")

function khatri_rao(A::AbstractMatrix, B::AbstractMatrix)
    size(A, 2) == size(B, 2) || error("Khatri-Rao requires same number of␣
 ↪columns.")
    m, r = size(A)
    n, _ = size(B)
    KR = zeros(promote_type(eltype(A), eltype(B)), m * n, r)
    for j in 1:r
        # Ordering matches unfold3 definitions below.
```

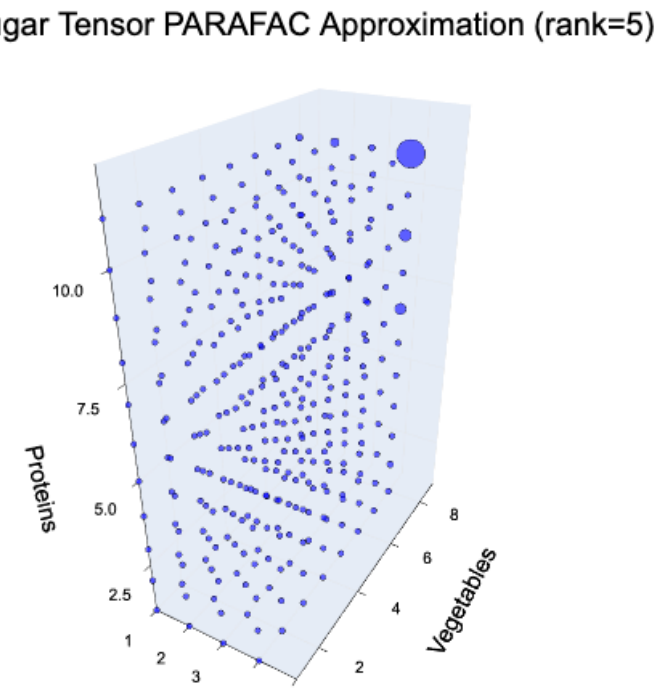
```
        KR[:, j] = vec(kron(A[:, j], B[:, j]))
    end
    return KR
end

function unfold3(X::Array{Float64,3}, mode::Int)
    if mode == 1
        return reshape(permutedims(X, (1, 2, 3)), size(X, 1), :)
    elseif mode == 2
        return reshape(permutedims(X, (2, 1, 3)), size(X, 2), :)
    elseif mode == 3
        return reshape(permutedims(X, (3, 1, 2)), size(X, 3), :)
    else
        error("mode must be 1, 2, or 3")
    end
end

function cp_reconstruct(A::Matrix{Float64}, B::Matrix{Float64}, C::
 ↪Matrix{Float64}, lambda::Vector{Float64})
    I1, R = size(A)
    J1, _ = size(B)
    K1, _ = size(C)
    Xhat = zeros(Float64, I1, J1, K1)
    for r in 1:R
        Xhat .+= lambda[r] .* reshape(A[:, r], I1, 1, 1) .* reshape(B[:, r], 1,␣
 ↪J1, 1) .* reshape(C[:, r], 1, 1, K1)
    end
    return Xhat
end

function cp_als_3way(X::Array{Float64,3}, R::Int; maxiter::Int=200, tol::
 ↪Float64=1e-7, seed::Int=1234, ridge::Float64=1e-8)
    I1, J1, K1 = size(X)
    Random.seed!(seed)

    # Positive initialization usually behaves better for nonnegative survey␣
 ↪tensors.
    A = rand(I1, R)
    B = rand(J1, R)
    C = rand(K1, R)
    lambda = ones(Float64, R)

    X1 = unfold3(X, 1)
    X2 = unfold3(X, 2)
    X3 = unfold3(X, 3)

    prev_relerr = Inf
```

```
    relerr = Inf

    for _ in 1:maxiter
        KRcb = khatri_rao(C, B)
        G1 = (B' * B) .* (C' * C) + ridge * Matrix{Float64}(LinearAlgebra.I, R,␣
 ↪R)
        A = (X1 * KRcb) / G1

        KRca = khatri_rao(C, A)
        G2 = (A' * A) .* (C' * C) + ridge * Matrix{Float64}(LinearAlgebra.I, R,␣
 ↪R)
        B = (X2 * KRca) / G2

        KRba = khatri_rao(B, A)
        G3 = (A' * A) .* (B' * B) + ridge * Matrix{Float64}(LinearAlgebra.I, R,␣
 ↪R)
        C = (X3 * KRba) / G3

        # Normalize columns each iteration and absorb scales in lambda.
        lambda .= 1.0
        for r in 1:R
            nA = norm(@view A[:, r]); nA = nA > 0 ? nA : 1.0
            nB = norm(@view B[:, r]); nB = nB > 0 ? nB : 1.0
            nC = norm(@view C[:, r]); nC = nC > 0 ? nC : 1.0
            A[:, r] ./= nA
            B[:, r] ./= nB
            C[:, r] ./= nC
            lambda[r] = nA * nB * nC
        end

        Xhat = cp_reconstruct(A, B, C, lambda)
        relerr = norm(X - Xhat) / max(norm(X), eps(Float64))

        if abs(prev_relerr - relerr) < tol
            break
        end
        prev_relerr = relerr
    end

    return A, B, C, lambda, relerr
end

# Build a dense copy for CP-ALS, then map the reconstruction back to ITensor␣
 ↪for plotting.
sugar_inds = collect(ITensors.inds(Sugar))
I = ITensors.dim(sugar_inds[1])
J = ITensors.dim(sugar_inds[2])
```

```
K = ITensors.dim(sugar_inds[3])

X_sugar = zeros(Float64, I, J, K)
for i in 1:I, j in 1:J, k in 1:K
    X_sugar[i, j, k] = Sugar[sugar_inds[1]=>i, sugar_inds[2]=>j,␣
 ↪sugar_inds[3]=>k]
end

cp_rank = 5
A_cp, B_cp, C_cp, lambda_cp, relerr_cp = cp_als_3way(X_sugar, cp_rank;␣
 ↪maxiter=300, tol=1e-8, seed=42, ridge=1e-7)
Xhat_cp = cp_reconstruct(A_cp, B_cp, C_cp, lambda_cp)

Sugar_parafac = ITensor(sugar_inds...)
for i in 1:I, j in 1:J, k in 1:K
    Sugar_parafac[sugar_inds[1]=>i, sugar_inds[2]=>j, sugar_inds[3]=>k] =␣
 ↪Xhat_cp[i, j, k]
end

println("PARAFAC rank: $cp_rank")
println("PARAFAC relative reconstruction error: $(round(relerr_cp, digits=6))")

plot_tensor(
    Sugar_parafac,
    title="Sugar Tensor PARAFAC Approximation (rank=$cp_rank)",
    xlabel=mode_label(sugar_inds[1]),
    ylabel=mode_label(sugar_inds[2]),
    zlabel=mode_label(sugar_inds[3])
)
```

```
PARAFAC rank: 5
PARAFAC relative reconstruction error: 0.005782
```

[25]:



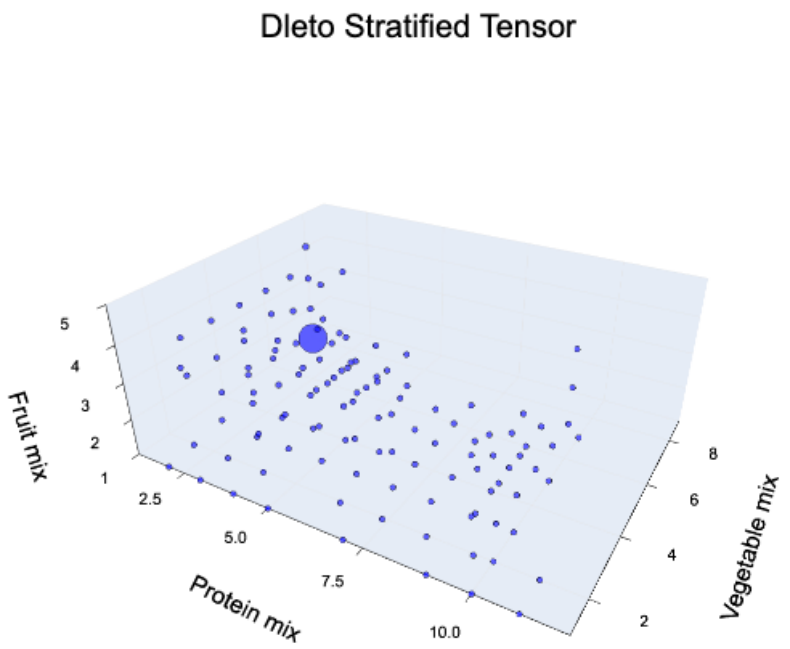
2 Stratification

We no run Dleto chiseling algorithms. First with the universal chisel, then with a hint at what we find an adjoint chisel. By rotating the image one can identify the generic stratification algorithm has uncovered a cluster that which is 1 x 1 in the

[26]:
```
@time Sugar_strat, Xs_strat = stratify(nondeg_Sugar)
inds_strat = collect(ITensors.inds(Sugar_strat))
plot_tensor(
    Sugar_strat,
    title="Dleto Stratified Tensor",
    xlabel=mode_label(inds_strat[1]; mixed=true),
    ylabel=mode_label(inds_strat[2]; mixed=true),
    zlabel=mode_label(inds_strat[3]; mixed=true)
 )
```

```
[ Info: Found 7 derivations for
stratification.

  5.108246 seconds (29.98 M allocations: 1.450 GiB, 3.59% gc time, 98.96%
compilation time: 10% of which was recompilation)
```

[26]:



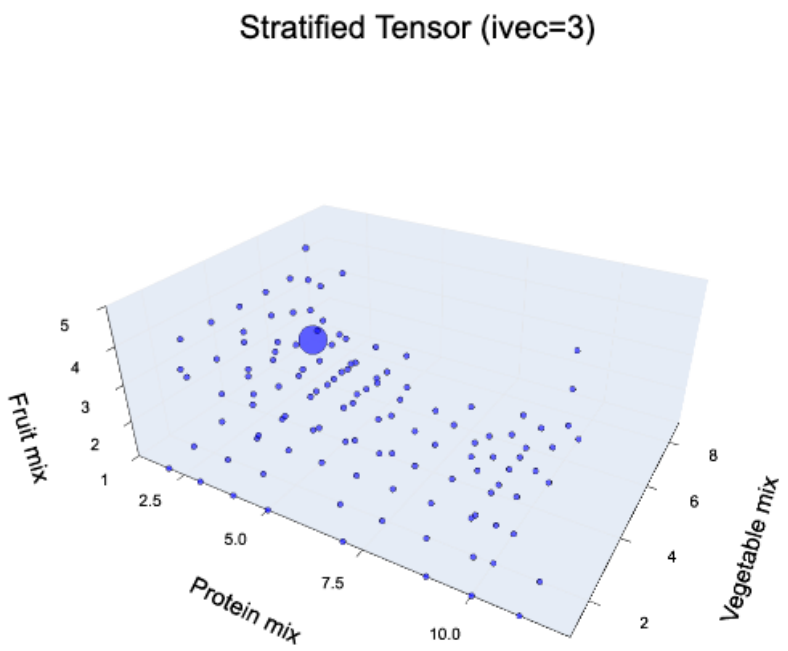
We see in fact that there 7 derivations. For a 3 tensor 2 are always trivial so we have 5 possible independent derivations to explore features. The first displayed already we can look at the others.

[27]:
```
# Explore derivation vectors ivec = 3:7 one by one and store results for reuse.
ch = UniversalChisel(length(ITensors.inds(nondeg_Sugar)))
fr = collect(ITensors.inds(nondeg_Sugar))
Ω = IndTransverseOps(fr, UniversalOp())

Sugar_strat_by_ivec = Dict{Int, ITensor}()
Xs_strat_by_ivec = Dict{Int, Vector{ITensor}}()

for ivec in 3:7
    println("\n--- Stratification with ivec = $ivec ---")
    Sugar_strat_i, Xs_strat_i = stratify(Ω, ch, nondeg_Sugar; ivec=ivec)

    # Store outputs so later cells can access them by ivec key.
    Sugar_strat_by_ivec[ivec] = Sugar_strat_i
    Xs_strat_by_ivec[ivec] = Xs_strat_i

    inds_i = collect(ITensors.inds(Sugar_strat_i))
    display(
        plot_tensor(
```

```
            Sugar_strat_i,
            title="Stratified Tensor (ivec=$ivec)",
            xlabel=mode_label(inds_i[1]; mixed=true),
            ylabel=mode_label(inds_i[2]; mixed=true),
            zlabel=mode_label(inds_i[3]; mixed=true)
        )
    )
end

println("Stored ivec keys: $(collect(keys(Sugar_strat_by_ivec)))")
```

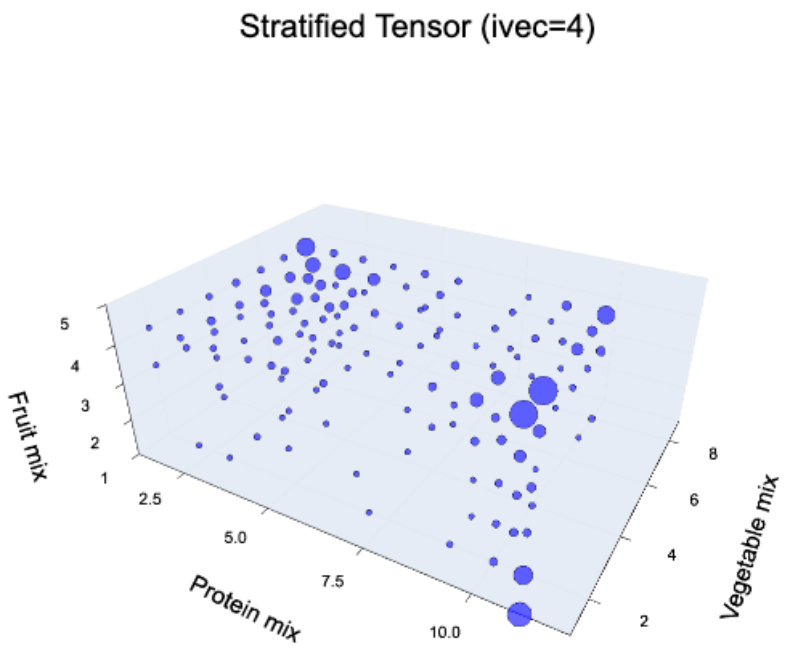
```
--- Stratification with ivec = 3 ---
```

```
[ Info: Found 7 derivations for
stratification.
```

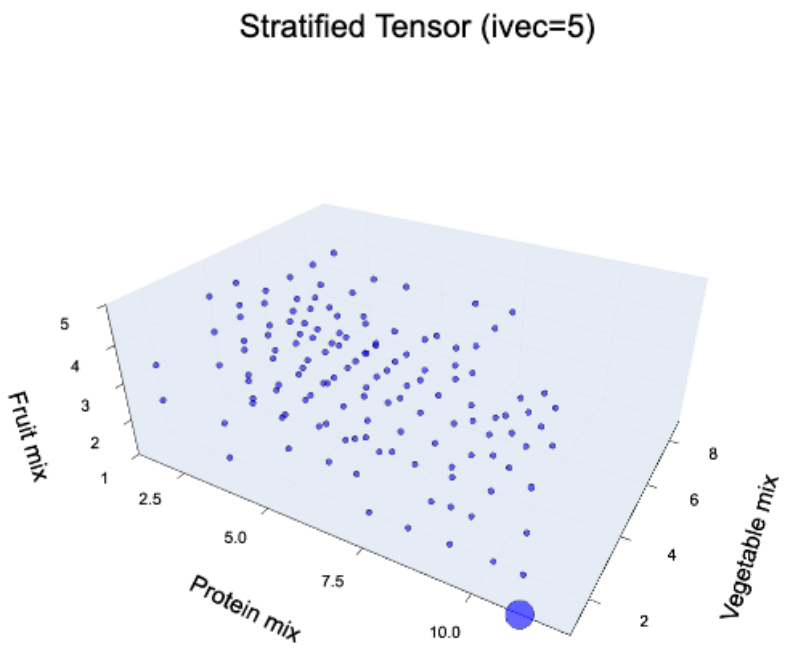



```
--- Stratification with ivec = 4 ---
```

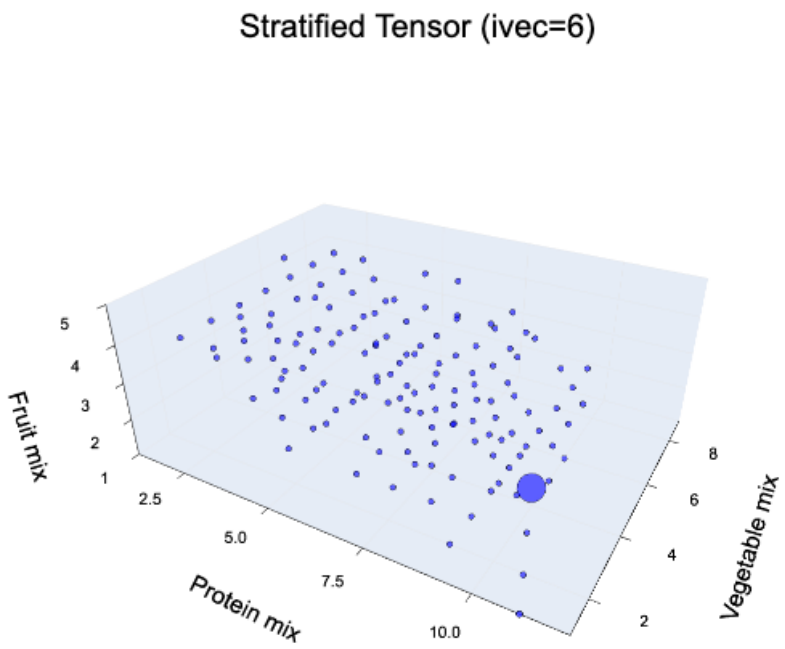
```
[ Info: Found 7 derivations for
stratification.
```



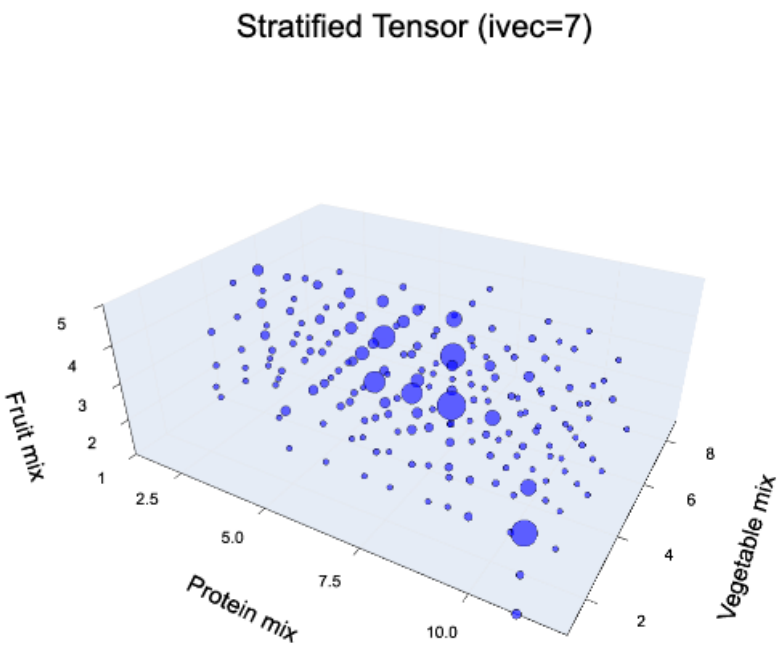

```
--- Stratification with ivec = 5 ---
[ Info: Found 7 derivations for
stratification.
```




```
--- Stratification with ivec = 6 ---
[ Info: Found 7 derivations for
stratification.
```




```
--- Stratification with ivec = 7 ---
[ Info: Found 7 derivations for
stratification.
```




```
Stored ivec keys: [5, 4, 6, 7, 3]
```

Let us now print out the mixtures. Remember this is an analytic probe so we allow negative coefficients.

[28]:
```
# Pivoted tables for ivec=3..7 using fixed cluster coordinates.
# Output focused on protein and vegetable modes (fruit omitted from reporting).
ivec_list = collect(3:7)

function fixed_cluster_coord_and_value(T::ITensor, iv::Int)
    inds_mix = collect(ITensors.inds(T))
    dims_mix = [ITensors.dim(i) for i in inds_mix]

    # Locate positions by dimension convention: 4=fruit, 8=veg, 11=protein.
    pos_fruit = findfirst(==(4), dims_mix)
    pos_veg = findfirst(==(8), dims_mix)
    pos_prot = findfirst(==(11), dims_mix)
    isnothing(pos_fruit) && error("Could not locate fruit mode (dim=4).")
    isnothing(pos_veg) && error("Could not locate vegetable mode (dim=8).")
    isnothing(pos_prot) && error("Could not locate protein mode (dim=11).")

    req_p, req_v = iv == 3 ? (1, 9) : (11, 1)
```

```
    prot_idx = clamp(req_p, 1, dims_mix[pos_prot])
    veg_idx = clamp(req_v, 1, dims_mix[pos_veg])

    # Pick fruit coordinate by largest magnitude on the fixed (p,v) slice.
    best_fruit = 1
    best_val = -Inf
    for fi in 1:dims_mix[pos_fruit]
        coord = [1, 1, 1]
        coord[pos_fruit] = fi
        coord[pos_veg] = veg_idx
        coord[pos_prot] = prot_idx
        vabs = abs(T[inds_mix[1]=>coord[1], inds_mix[2]=>coord[2],␣
 ↪inds_mix[3]=>coord[3]])
        if vabs > best_val
            best_val = vabs
            best_fruit = fi
        end
    end

    best_coord = [1, 1, 1]
    best_coord[pos_fruit] = best_fruit
    best_coord[pos_veg] = veg_idx
    best_coord[pos_prot] = prot_idx

    return inds_mix, dims_mix, Tuple(best_coord), best_val, req_p, req_v,␣
 ↪prot_idx, veg_idx
end

function print_pivot(categories::Vector{String}, value_by_cat::
 ↪Dict{String,Dict{Int,Float64}}, ivecs::Vector{Int};␣
 ↪value_fmt=x->string(round(x, digits=1), "%"))
    headers = vcat(["category"], ["ivec=$iv" for iv in ivecs])

    rows = Vector{Vector{String}}()
    for cat in categories
        row = String[cat]
        for iv in ivecs
            v = get(get(value_by_cat, cat, Dict{Int,Float64}()), iv, NaN)
            push!(row, isnan(v) ? "" : value_fmt(v))
        end
        push!(rows, row)
    end

    widths = [length(h) for h in headers]
    for row in rows
        for j in eachindex(row)
            widths[j] = max(widths[j], length(row[j]))
```

```
        end
    end

    println(join([rpad(headers[j], widths[j]) for j in eachindex(headers)], " |␣
 ↪"))
    println(join([repeat("-", w) for w in widths], "-+-"))
    for row in rows
        println(join([rpad(row[j], widths[j]) for j in eachindex(row)], " | "))
    end
end

summary_abs = Dict{Int,Float64}()
summary_pv = Dict{Int,String}()
protein_by_cat = Dict{String,Dict{Int,Float64}}()
veg_by_cat = Dict{String,Dict{Int,Float64}}()

protein_order = [friendly_label(name) for name in proteins_with_empty]
veg_order = [friendly_label(name) for name in vegetables_with_empty]

for iv in ivec_list
    haskey(Sugar_strat_by_ivec, iv) || error("Missing Sugar_strat_by_ivec[$iv].␣
 ↪Run the ivec storage cell.")
    haskey(Xs_strat_by_ivec, iv) || error("Missing Xs_strat_by_ivec[$iv]. Run␣
 ↪the ivec storage cell.")

    Sugar_strat_iv = Sugar_strat_by_ivec[iv]
    Xs_iv = Xs_strat_by_ivec[iv]

    inds_mix, dims_mix, best_coord, best_val, req_p, req_v, prot_idx, veg_idx =␣
 ↪fixed_cluster_coord_and_value(Sugar_strat_iv, iv)
    summary_abs[iv] = best_val
    summary_pv[iv] = "(p=>$prot_idx, v=>$veg_idx)"

    pos_prot = findfirst(==(11), dims_mix)
    pos_veg = findfirst(==(8), dims_mix)

    X_prot = first(X for X in Xs_iv if any(ITensors.dim(ix) == 11 for ix in␣
 ↪ITensors.inds(X)))
    X_veg = first(X for X in Xs_iv if any(ITensors.dim(ix) == 8 for ix in␣
 ↪ITensors.inds(X)))

    coord_prot = best_coord[pos_prot]
    coord_veg = best_coord[pos_veg]

    mix_prot_idx = inds_mix[pos_prot]
    mix_veg_idx = inds_mix[pos_veg]
```

```
    e_prot = ITensor(mix_prot_idx); e_prot[mix_prot_idx=>coord_prot] = 1.0
    e_veg = ITensor(mix_veg_idx); e_veg[mix_veg_idx=>coord_veg] = 1.0

    prot_mix_old = dag(X_prot) * e_prot
    veg_mix_old = dag(X_veg) * e_veg

    prot_old_idx = only([ix for ix in ITensors.inds(prot_mix_old) if ITensors.
 ↪dim(ix) == 11])
    veg_old_idx = only([ix for ix in ITensors.inds(veg_mix_old) if ITensors.
 ↪dim(ix) == 8])

    prot_weights = [prot_mix_old[prot_old_idx=>i] for i in 1:ITensors.
 ↪dim(prot_old_idx)]
    veg_weights = [veg_mix_old[veg_old_idx=>i] for i in 1:ITensors.
 ↪dim(veg_old_idx)]

    for (name, w) in zip(proteins_with_empty, prot_weights)
        abs(w) > 1e-3 || continue
        cat = friendly_label(name)
        get!(protein_by_cat, cat, Dict{Int,Float64}())[iv] = 100 * w
    end

    for (name, w) in zip(vegetables_with_empty, veg_weights)
        abs(w) > 1e-3 || continue
        cat = friendly_label(name)
        get!(veg_by_cat, cat, Dict{Int,Float64}())[iv] = 100 * w
    end
end

println("\nFixed Cluster Summary (protein/veg only)")
coord_rows = [
    ["selected (p,v)"; [summary_pv[iv] for iv in ivec_list]...],
    ["|value|"; [string(round(summary_abs[iv], digits=4)) for iv in ivec_list]..
 ↪.]
]
coord_headers = vcat(["metric"], ["ivec=$iv" for iv in ivec_list])
coord_widths = [length(h) for h in coord_headers]
for row in coord_rows
    for j in eachindex(row)
        coord_widths[j] = max(coord_widths[j], length(row[j]))
    end
end
println(join([rpad(coord_headers[j], coord_widths[j]) for j in␣
 ↪eachindex(coord_headers)], " | "))
println(join([repeat("-", w) for w in coord_widths], "-+-"))
```

```
for row in coord_rows
    println(join([rpad(row[j], coord_widths[j]) for j in eachindex(row)], " |␣
 ↪"))
end

println("\nProtein Mixtures (%; ivec in columns)")
protein_categories = [c for c in protein_order if haskey(protein_by_cat, c)]
print_pivot(protein_categories, protein_by_cat, ivec_list)

println("\nVegetable Mixtures (%; ivec in columns)")
veg_categories = [c for c in veg_order if haskey(veg_by_cat, c)]
print_pivot(veg_categories, veg_by_cat, ivec_list)
```

```
Fixed Cluster Summary (protein/veg only)
metric          | ivec=3        | ivec=4         | ivec=5         | ivec=6         |
ivec=7
----------------+---------------+----------------+----------------+----------------+-
-------------
selected (p,v) | (p=>1, v=>8) | (p=>11, v=>1) | (p=>11, v=>1) | (p=>11, v=>1) |
(p=>11, v=>1)
|value|         | 0.0757        | 0.0022         | 1.0629         | 1.128          |
0.0049

Protein Mixtures (%; ivec in columns)
category                       | ivec=3 | ivec=4 | ivec=5 | ivec=6 | ivec=7
-------------------------------+--------+--------+--------+--------+-------
Meat (beef, pork, lamb, game) | 36.1%  | 16.3%  | -15.2% | -5.5%  | -19.0%
Cured meat                     | 15.7%  | -14.5% | -9.4%  | 11.9%  | 13.6%
Organ meats                    | 4.3%   | 1.1%   | -75.9% | -84.6% | 1.1%
Poultry                        | 38.3%  | 10.1%  | -17.3% | -1.4%  | -12.1%
High omega-3 seafood           | 1.7%   | -1.7%  | -19.3% | -19.5% | 2.1%
Low omega-3 seafood            | 12.6%  | -7.0%  | -7.4%  | 5.9%   | 5.6%
Eggs                           | 65.2%  | 50.4%  | -21.3% | -20.0% | -53.9%
Soy products                   | -0.8%  | 2.0%   | -9.6%  | -13.2% | -1.4%
Nuts and seeds                 | 40.2%  | -74.4% | -44.4% | 37.3%  | 71.9%
Legumes (as protein)           | -13.3% | 34.7%  | 18.6%  | -16.0% | -33.9%
No measurable protein          | -27.4% | 8.5%   | 17.9%  | -5.4%  | -7.0%

Vegetable Mixtures (%; ivec in columns)
category                         | ivec=3 | ivec=4 | ivec=5 | ivec=6 | ivec=7
---------------------------------+--------+--------+--------+--------+-------
Dark green vegetables            | 74.5%  | 74.5%  | -74.5% | 74.5%  | -74.5%
Red/orange vegetables (tomatoes) | 0.3%   | 0.3%   | -0.3%  | 0.3%   |
Red/orange vegetables (other)    | -1.7%  | -1.7%  | 1.7%   | -1.7%  | 1.6%
Starchy vegetables (potatoes)    | -53.7% | -53.7% | 53.7%  | -53.7% | 53.7%
Starchy vegetables (other)       | 39.5%  | 39.5%  | -39.5% | 39.5%  | -39.5%
Other vegetables                 | -1.1%  | -1.1%  | 1.1%   | -1.1%  | 0.5%
```

```
Legumes (as vegetables)          | 0.4%   | 0.4%   | -0.4%  | 0.4%   | -0.5%
No measurable vegetable          |        |        |        |        | 0.8%
```

3 Adjoint Chisel Stratification

Since we saw the most obvious clustering in the fruit-vegetable axes we next design a chisel to focus only on these. This is a version of what is known as an *Adjoint Chisel*.

[29]:
```
# Use an adjoint chisel on the fruit/vegetable axes so the protein axis stays␣
 ↪at 0.
ch = AdjointChisel(3, 1, 2)
fr = collect(ITensors.inds(nondeg_Sugar))
Ω = IndTransverseOps(fr, UniversalOp())

@time Sugar_adj_strat, Xs_strat = stratify(Ω, ch, nondeg_Sugar; tol=1e-6);
```

```
  0.027983 seconds (245.98 k allocations: 26.109 MiB, 58.95% compilation time)
[ Info: Found 4 derivations for
stratification.
```

[30]:
```
# Plot the nontrivial adjoint-chisel derivations ivec = 2, 3, 4,
# then print protein/vegetable vector tables for requested fixed clusters.
ch_adj = AdjointChisel(3, 1, 2)
fr_adj = collect(ITensors.inds(nondeg_Sugar))
Ω_adj = IndTransverseOps(fr_adj, UniversalOp())

adj_by_ivec = Dict{Int,ITensor}()
adj_Xs_by_ivec = Dict{Int,Vector{ITensor}}()

for ivec in 2:4
    println("\n--- Adjoint stratification with ivec = $ivec ---")
    Sugar_adj_i, Xs_adj_i = stratify(Ω_adj, ch_adj, nondeg_Sugar; tol=1e-6,␣
 ↪ivec=ivec)
    adj_by_ivec[ivec] = Sugar_adj_i
    adj_Xs_by_ivec[ivec] = Xs_adj_i

    inds_adj_i = collect(ITensors.inds(Sugar_adj_i))
    display(
        plot_tensor(
            Sugar_adj_i,
            title="Adjoint Stratified Tensor (ivec=$ivec)",
            xlabel=mode_label(inds_adj_i[1]; mixed=true),
            ylabel=mode_label(inds_adj_i[2]; mixed=true),
            zlabel=mode_label(inds_adj_i[3]; mixed=true)
        )
    )
end
```

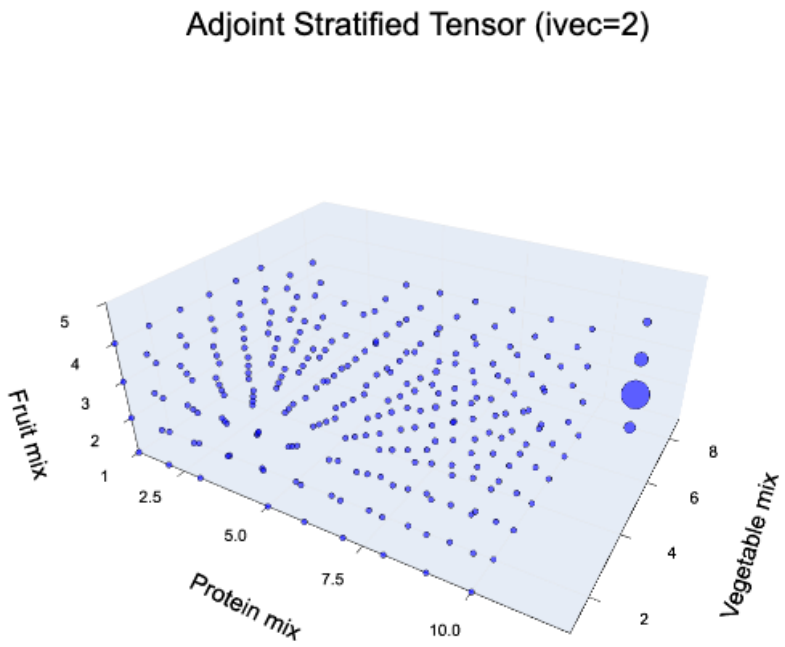
```
# Fixed cluster coordinates requested by user:
# ivec=2 and 4: (p=>11, v=>8), ivec=3: (p=>1, v=>1)
fixed_req = Dict(2 => (11, 8), 3 => (1, 1), 4 => (11, 8))

function print_pivot(categories::Vector{String}, value_by_cat::
 ↪Dict{String,Dict{Int,Float64}}, ivecs::Vector{Int};␣
 ↪value_fmt=x->string(round(x, digits=1), "%"))
    headers = vcat(["category"], ["ivec=$iv" for iv in ivecs])

    rows = Vector{Vector{String}}()
    for cat in categories
        row = String[cat]
        for iv in ivecs
            v = get(get(value_by_cat, cat, Dict{Int,Float64}()), iv, NaN)
            push!(row, isnan(v) ? "" : value_fmt(v))
        end
        push!(rows, row)
    end

    widths = [length(h) for h in headers]
    for row in rows
        for j in eachindex(row)
            widths[j] = max(widths[j], length(row[j]))
        end
    end

    println(join([rpad(headers[j], widths[j]) for j in eachindex(headers)], " |␣
 ↪"))
    println(join([repeat("-", w) for w in widths], "-+-"))
    for row in rows
        println(join([rpad(row[j], widths[j]) for j in eachindex(row)], " | "))
    end
end

ivec_list = [2, 3, 4]
protein_by_cat = Dict{String,Dict{Int,Float64}}()
veg_by_cat = Dict{String,Dict{Int,Float64}}()
protein_order = [friendly_label(name) for name in proteins_with_empty]
veg_order = [friendly_label(name) for name in vegetables_with_empty]
summary_rows = NamedTuple[]

for ivec in ivec_list
    T = adj_by_ivec[ivec]
    Xs = adj_Xs_by_ivec[ivec]
    inds_mix = collect(ITensors.inds(T))
    dims_mix = [ITensors.dim(i) for i in inds_mix]
```

```
  pos_veg = findfirst(==(8), dims_mix)
  pos_prot = findfirst(==(11), dims_mix)
  isnothing(pos_veg) && error("Could not locate vegetable mode (dim=8) for␣
↪ivec=$ivec.")
  isnothing(pos_prot) && error("Could not locate protein mode (dim=11) for␣
↪ivec=$ivec.")

  req_p, req_v = fixed_req[ivec]
  prot_idx = clamp(req_p, 1, dims_mix[pos_prot])
  veg_idx = clamp(req_v, 1, dims_mix[pos_veg])

  X_prot = first(X for X in Xs if any(ITensors.dim(ix) == 11 for ix in␣
↪ITensors.inds(X)))
  X_veg = first(X for X in Xs if any(ITensors.dim(ix) == 8 for ix in ITensors.
↪inds(X)))

  mix_prot_idx = inds_mix[pos_prot]
  mix_veg_idx = inds_mix[pos_veg]

  # Use fixed requested (p,v) coordinates directly for vector reconstruction.
  e_prot = ITensor(mix_prot_idx); e_prot[mix_prot_idx=>prot_idx] = 1.0
  e_veg = ITensor(mix_veg_idx); e_veg[mix_veg_idx=>veg_idx] = 1.0

  prot_mix_old = dag(X_prot) * e_prot
  veg_mix_old = dag(X_veg) * e_veg

  prot_old_idx = only([ix for ix in ITensors.inds(prot_mix_old) if ITensors.
↪dim(ix) == 11])
  veg_old_idx = only([ix for ix in ITensors.inds(veg_mix_old) if ITensors.
↪dim(ix) == 8])

  prot_weights = [prot_mix_old[prot_old_idx=>i] for i in 1:ITensors.
↪dim(prot_old_idx)]
  veg_weights = [veg_mix_old[veg_old_idx=>i] for i in 1:ITensors.
↪dim(veg_old_idx)]

  for (name, w) in zip(proteins_with_empty, prot_weights)
      abs(w) > 1e-3 || continue
      cat = friendly_label(name)
      get!(protein_by_cat, cat, Dict{Int,Float64}())[ivec] = 100 * w
  end

  for (name, w) in zip(vegetables_with_empty, veg_weights)
      abs(w) > 1e-3 || continue
      cat = friendly_label(name)
```

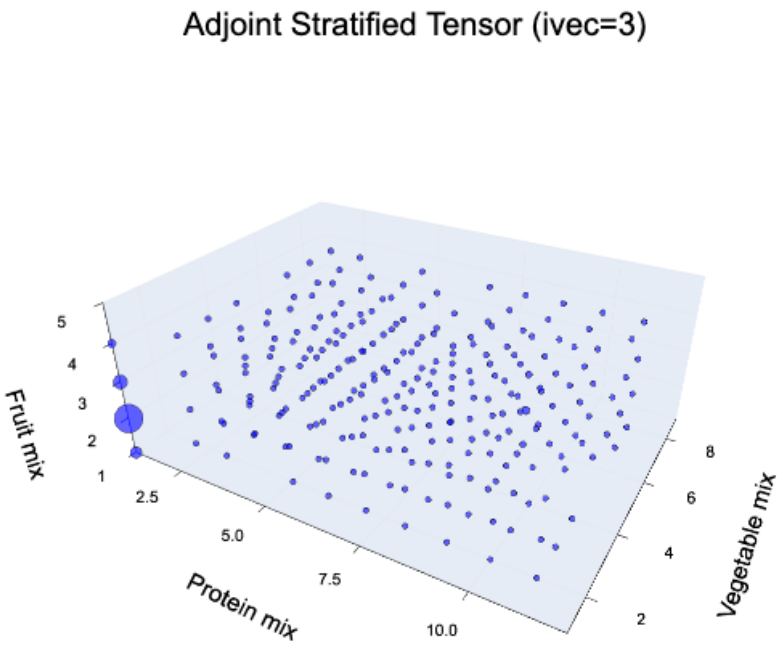
```
        get!(veg_by_cat, cat, Dict{Int,Float64}())[ivec] = 100 * w
    end

    push!(summary_rows, (
        ivec = ivec,
        requested = "(p=>$req_p, v=>$req_v)",
        used = "(p=>$prot_idx, v=>$veg_idx)"
    ))
end

# Print summary + vector tables after the three 3D plots.
headers = ["ivec", "requested", "used"]
getters = [r -> r.ivec, r -> r.requested, r -> r.used]
widths = [length(h) for h in headers]
for row in summary_rows
    for (j, g) in enumerate(getters)
        widths[j] = max(widths[j], length(string(g(row))))
    end
end

println("\nFixed Cluster Table (Adjoint ivec=2,3,4)")
println(join([rpad(headers[j], widths[j]) for j in eachindex(headers)], " | "))
println(join([repeat("-", w) for w in widths], "-+-"))
for row in summary_rows
    println(join([rpad(string(getters[j](row)), widths[j]) for j in␣
 ↪eachindex(getters)], " | "))
end

println("\nProtein Vectors (%; ivec in columns)")
protein_categories = [c for c in protein_order if haskey(protein_by_cat, c)]
print_pivot(protein_categories, protein_by_cat, ivec_list)

println("\nVegetable Vectors (%; ivec in columns)")
veg_categories = [c for c in veg_order if haskey(veg_by_cat, c)]
print_pivot(veg_categories, veg_by_cat, ivec_list)
```

```
--- Adjoint stratification with ivec = 2 ---
```

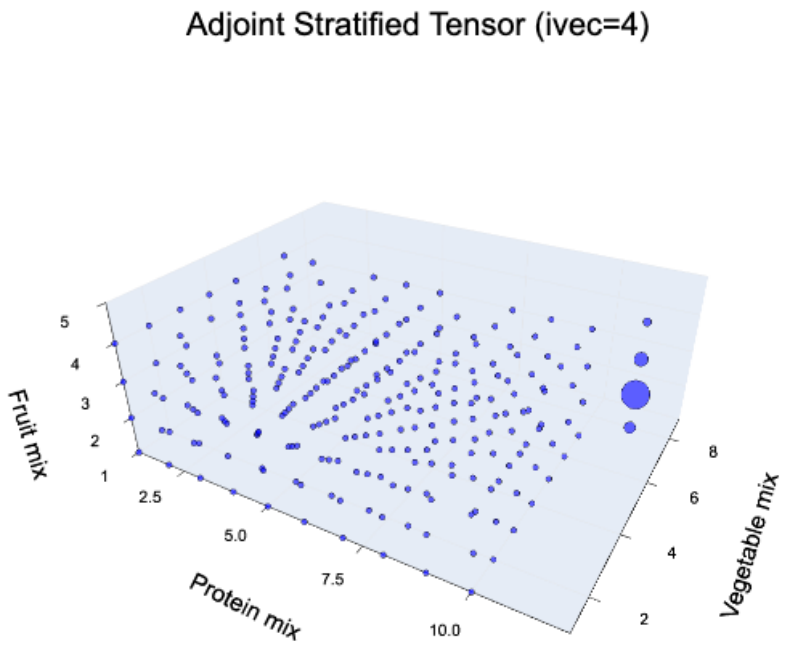
```
[ Info: Found 4 derivations for
stratification.
```




```
--- Adjoint stratification with ivec = 3 ---
[ Info: Found 4 derivations for
stratification.
```




```
--- Adjoint stratification with ivec = 4 ---
[ Info: Found 4 derivations for
stratification.
```



Fixed Cluster Table (Adjoint ivec=2,3,4)

ivec	requested	used
2	(p=>11, v=>8)	(p=>11, v=>8)
3	(p=>1, v=>1)	(p=>1, v=>1)
4	(p=>11, v=>8)	(p=>11, v=>8)

Protein Vectors (%; ivec in columns)

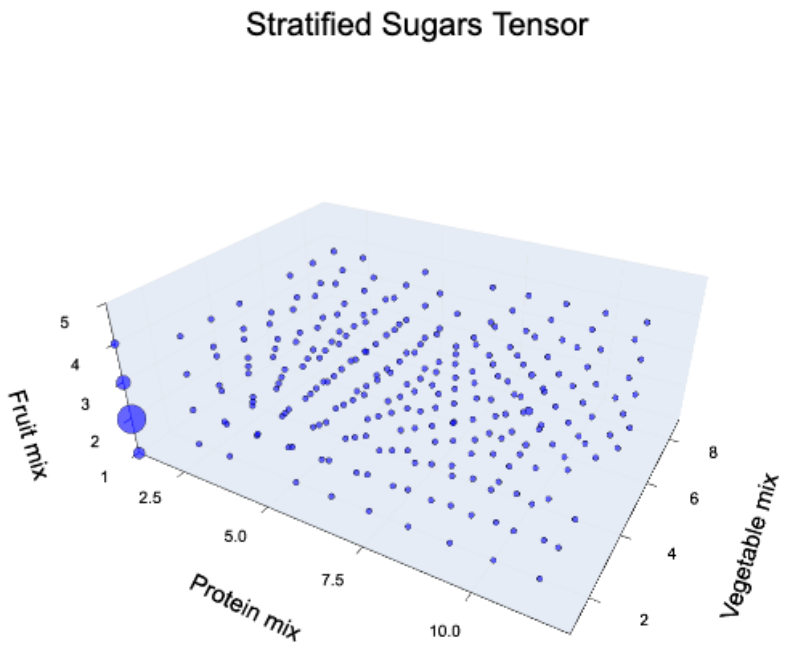
category	ivec=2	ivec=3	ivec=4
Meat (beef, pork, lamb, game)	30.1%	25.5%	2.4%
Cured meat	19.7%	-9.2%	-4.2%
Organ meats	-3.6%	-3.6%	96.0%
Poultry	34.1%	20.3%	1.2%
High omega-3 seafood	0.4%	-2.5%	23.2%
Low omega-3 seafood	14.3%	-3.1%	-1.6%
Eggs	48.2%	66.1%	5.0%
Soy products	-2.4%	0.8%	13.8%
Nuts and seeds	60.4%	-59.9%	0.6%
Legumes (as protein)	-22.8%	29.5%	-1.2%
No measurable protein	-28.9%	0.6%	-1.6%

```
Vegetable Vectors (%; ivec in columns)
category                          | ivec=2 | ivec=3 | ivec=4
----------------------------------+--------+--------+-------
Dark green vegetables             | 74.5%  | 74.5%  | 74.5%
Red/orange vegetables (tomatoes)  | 0.3%   | 0.3%   | 0.3%
Red/orange vegetables (other)     | -1.7%  | -1.7%  | -1.7%
Starchy vegetables (potatoes)     | -53.7% | -53.7% | -53.7%
Starchy vegetables (other)        | 39.5%  | 39.5%  | 39.5%
Other vegetables                  | -1.1%  | -1.1%  | -1.1%
Legumes (as vegetables)           | 0.4%   | 0.4%   | 0.4%
```

Adjoints have 1 trivial solution so even in the adjoint position there 3 feature vectors (less than the 5 for the universal chisel).

[31]:
```
inds_adj = collect(ITensors.inds(Sugar_adj_strat))
plot_tensor(Sugar_adj_strat, title="Stratified Sugars Tensor",
        xlabel=mode_label(inds_adj[1]; mixed=true),␣
 ↪ylabel=mode_label(inds_adj[2]; mixed=true), zlabel=mode_label(inds_adj[3];␣
 ↪mixed=true))
```

[31]:



3.1 Summary

While this experiment is for demonstration purposes and we are not including subject mater expertise we can draw the following conclusions form the computation.

1. Tucker decompositions at the default tolerance are trivial.
2. Higher Order SVDs do not appear to locate a specific cluster in fact they make the data geometrically more dense than in the given representation.
3. The Dleto universal chisel identifies 5 independent derivations. Their applied null patterns identify a block in the protein-vegetable modes. Further clustering structure is visible but less detached but hints at more nuanced clustering.
4. The clustering is visible even in the adjoint chisel but with only 3 independent derivations so it features more structure but not as much as with universal chiselling.
5. Analyzing the main cluster, the fruit mode is ignored, the mixtures supports combinations of

- proteins that are strong in meat, cured meat, and poultry;
- vegetables that are a strong mix of dark greens and starchy vegetables that are not potatoes.

Without subject matter knowledge we can only guess what is the source of the added sugars in these foods. Perhaps these are in the form of salad dressings or sauces? We can only conjecture. The point is however that the data here encodes a signal to pursue that is evident with the chiselling method and at least not transparently noticed by other rank related strategies. Since the outlier here is small dimension (1-dimensional) it is understandable that its influence on singles such as singular values will be muted.